Predicting Quarterly Airbnb Booking Counts in New York City with Tree-Based Ensembles and a Weighted Model Blend

Muhammed Ali Bakır and Mahmut Sami Başkal [1] [2]

Abstract

Forecasting how much a short-term rental will be booked is a useful but difficult problem for both hosts and the platforms that connect them with guests. In this paper we present the final report of our COMP468 (Machine Learning in Python) project at Abdullah Gül University. The task, hosted on Kaggle as the "A Cloned Airbnb Prediction Competition – K353," is to predict the total number of nights booked in the third quarter of 2016 for around 24,000 Airbnb properties in New York City; submissions are ranked by Mean Squared Error (MSE). Starting from five raw CSV files (one static property table and two quarters of daily listing histories totalling more than eight million rows), we engineered a flat matrix of 150 features per property covering price dynamics, availability and booking behaviour, status churn, recency, a quarter-over-quarter demand proxy, text keywords from listing titles, and geographic clusters. We then compared five models taught in the course—Linear Regression, Random Forest, Gradient Boosting, XGBoost, and a PyTorch Multi-Layer Perceptron (MLP) trained with Adam—inside leakage-safe scikit-learn pipelines, and tuned the boosted trees with a custom randomized search that uses fold-level early stopping. Our single best model, a retuned XGBoost regressor, reached a 5-fold cross-validation MSE of 182.2 on the local training set, and a convex (SLSQP) weighted blend lowered the out-of-fold MSE further to 180.0. We formally define the target-encoding, search, MLP, and blending objectives, and we verify the blend gain with a paired significance test ($t = 5.33$, $p = 1.0 \times 10^{-7}$) and a residual analysis on the bounded $[0,92]$ target. The blended submission scored 185.499 MSE on the Kaggle leaderboard, placing our team first out of six competing teams. We also discuss, in detail, which parts of our midterm plan we kept, which we dropped (the target log-transform, TF-IDF/topic features, and host-level encoding), and why the final pipeline took a different path.

***Index Terms*—Airbnb, demand forecasting, regression, ensemble learning, XGBoost, random** *forest, multi-layer perceptron, model blending, target encoding, constrained optimization, mean squared error, feature engineering.*

[1] Computer Engineering Department, Abdullah Gül University, Kayseri, Türkiye. E-mail: muhammedali.bakir@agu.edu.tr; mahmutsami.baskal@agu.edu.tr.

[2] This work was conducted as part of COMP468 (Machine Learning in Python) at Abdullah Gül University (AGU), instructed by Dr. Khaled Hejja.

I. Introduction

the last ten years, peer-to-peer (P2P) accommodation platforms have reshaped the way people travel and the way property owners earn money from their homes. Airbnb is the most visible example of this shift, with millions of active listings spread across more than a hundred thousand cities worldwide [1]. For a platform of that size, two questions tend to dominate the business: what is a fair price for a given listing, and how much demand will that listing actually receive over some future period. In our midterm draft we already framed the project around the second of those questions, and in this final report we keep that focus and carry it all the way through to a working, leaderboard-ranked solution.

Being able to predict demand a quarter in advance is valuable for several practical reasons. A host who knows that the next three months will be busy can schedule cleaning and maintenance, raise prices for the high-demand weeks, or decide that it is worth hiring help. The platform, on its side, can plan customer-support capacity, target marketing toward listings that look like they will be under-booked, and give hosts better pricing recommendations. The trouble is that demand is the result of many interacting factors—the property type and capacity, the neighbourhood, seasonality, the host's responsiveness, the review history, and how aggressively the price moves during the period. Classical linear models have been used for this kind of accommodation data [2], [3], but a fairly consistent finding in the recent literature is that machine-learning models, and tree-based ensembles in particular, tend to do better on the mixed numerical and categorical tables that describe short-term rentals [4], [5], [6].

Concretely, our team took part in the Kaggle competition "A Cloned Airbnb Prediction Competition – K353" [7]. The goal is to predict, for every property in a held-out New York City test set, the total number of nights booked during the third quarter (July, August, and September) of 2016. Because a quarter contains 92 days, the target is an integer between 0 and 92, and the competition scores submissions purely by Mean Squared Error. The whole project was constrained in one important way: every model and optimization algorithm we use has to come from the COMP468 course modules, so deliberately popular Kaggle tools such as CatBoost or LightGBM were off the table for us and we did not use them.

This report is organized as follows. Section 2 reviews the most relevant prior work on Airbnb prediction, decision-tree ensembles, hyper-parameter tuning, and preprocessing, and adds a short mathematical argument for why linear models struggle with a zero-inflated bounded target. Section 3 describes the project setting: the problem statement, the dataset, and the competition. Section 4 gives the full methodology and, in contrast to the midterm draft, now defines every custom component—the target encoder, the search loop, the MLP, and the blend—with explicit equations. Section 5 presents our results, a significance test for the blend, and a residual analysis on the target boundaries. Section 6 goes through what changed between the midterm plan and the final submission. Section 7 concludes and lists what we would try next.

II. Literature Review

We group the related work into four areas: (A) machine learning for Airbnb prediction tasks, (B) decision trees and tree-based ensembles, (C) hyper-parameter tuning and model validation, and (D) data preprocessing and feature engineering. Most of this review was already written for our midterm draft; here we keep the parts that ended up actually guiding the final solution and trim the ones that did not.

A. Machine Learning for Airbnb Prediction Tasks

Several studies apply machine learning directly to Airbnb data. Tang, Cheng, and Zhang [4] worked with listings from Sydney and compared ten algorithms including Random Forest, Gradient Boosting, and CatBoost for price prediction. CatBoost gave them the lowest root mean squared error, and the most influential features were the maximum number of guests, the number of bedrooms, and whether the room was private or shared. Camatti et al. [5] ran a comparative analysis of artificial-intelligence and traditional linear methods on Dutch Airbnb listings, and concluded that tree-based and neural-network methods overfit less than linear regression, though linear models are still handy for reading off which predictors matter. Rezazadeh Kalehbasti, Nikolenko, and Rezaei [6] predicted New York City Airbnb prices using property features, host characteristics, and the sentiment of customer reviews; their best model was a regularized linear model built on review-text features, but tree models such as Random Forest and Support Vector Regression were close behind.

Other work looks at demand rather than price, which is closer to our own task. Siker [8] tackled the Kaggle "Airbnb New User Bookings" problem with XGBoost, and even though the target there (destination country) is different from ours, the workflow—heavy feature engineering followed by gradient boosting—matches how most Airbnb-style Kaggle problems are solved in practice. Chapman, Mohammad, and Villegas [9] studied dynamic short-term rental markets and built a pipeline that merged listing, calendar, and review data; they again found gradient boosting beating linear regression. Islam et al. [10] combined a Latent Dirichlet Allocation topic model over property descriptions with an XGBoost regressor and improved price prediction on the San Jose dataset. Two lessons from this group shaped our project: first, the most useful predictors are usually a mix of structural features, location, and dynamic calendar/review information; and second, gradient-boosted trees are a strong default because they cope well with mixed feature types and missing values.

A recurring theme across these studies is that the engineering choices around the model often matter more than the model itself. Tang et al. [4] and Islam et al. [10] both report that their

largest gains came from the features they constructed—guest capacity and room type in the first case, topic structure of the listing description in the second—rather than from swapping one boosting library for another. Kalehbasti et al. [6] similarly found that review-sentiment features moved their error more than the choice between a regularized linear model and a tree ensemble. This is consistent with the broader observation that on tabular problems the ceiling is usually set by how much signal the practitioner can encode, and it is precisely why we invested most of our effort in the 150-feature matrix and treated the model comparison as a secondary, if still important, step. It also explains why a competition with a fixed, shared dataset is decided largely by feature engineering and validation discipline rather than by access to a particular algorithm—an observation that shaped how we allocated our time.

B. Why Linear Models Underfit a Zero-Inflated Bounded Target

Beyond the empirical observation that linear regression trails the trees, it is worth stating *why* in mathematical terms, because the shape of our target is the crux of the whole problem. Ordinary least squares fits $\hat{y}(\boldsymbol{x}) = \boldsymbol{\beta}^\top \boldsymbol{x}$ by minimizing the population risk $\mathbb{E}[(y - \boldsymbol{\beta}^\top \boldsymbol{x})^2]$, whose minimizer is the conditional mean $\mathbb{E}[y \mid \boldsymbol{x}]$ *restricted to the class of affine functions*. Our target, however, is a censored count confined to $[0,92]$ with a large probability mass at 0 (roughly 54% of the development rows are exactly zero; see Fig. 1). Its true conditional mean is therefore an inherently nonlinear, saturating function of the covariates: it must flatten to 0 for low-demand listings and saturate toward 92 for high-demand ones. An affine predictor cannot reproduce those two plateaus—it extrapolates linearly past both edges, so it produces negative fitted values for the dead listings (which are then clipped, wasting capacity) and undershoots the busiest ones. Formally, if $y = \max(0, \min(92, \boldsymbol{\beta}^\top \boldsymbol{x} + \varepsilon))$, the censoring makes $\mathbb{E}[y \mid \boldsymbol{x}]$ a *nonlinear* function of $\boldsymbol{x}$ even when the latent variable is linear, which is exactly the limited-dependent-variable setting that motivated Tobit-type estimators [11] and, more directly for counts, two-part (hurdle) models. Earlier price-prediction studies that reported linear baselines [2], [3] did not face this point mass because price is continuous and unbounded; demand counts are not, and that is the structural reason their linear formulations do not transfer to our task. Tree ensembles sidestep the issue entirely: a regression tree approximates the saturating conditional mean with axis-aligned piecewise-constant regions, so the 0 and 92 plateaus are simply leaves, and boosting refines the transition region between them.

C. Modelling Zero-Inflated and Count Targets

Our target is a bounded count with a large spike at zero, and the statistics literature offers a dedicated family of models for exactly this shape. A plain Poisson regression assumes the conditional mean equals the conditional variance, which is violated here because the zero spike and the upper-bound pile-up make the data strongly over-dispersed. Negative-binomial regression relaxes that equal-mean-variance assumption but still does not account for the excess of structural zeros. Two model classes target the zeros directly: zero-inflated models (e.g. zero-inflated Poisson, ZIP) mix a point mass at zero with a count distribution, and *hurdle* (two-part) models first decide whether the count is positive and then model the positive counts separately. Censored-regression estimators such as Tobit [11] instead treat the bound itself as the generative mechanism. We did not adopt these parametric estimators in the final pipeline—they are outside the COMP468 module list, and gradient-boosted trees already approximate the saturating conditional mean non-parametrically—but they frame the problem precisely and directly motivate the two-stage hurdle extension we propose in Section 7. The residual analysis in Section 5.5 shows empirically why a single least-squares regressor leaves room for such a two-part treatment: its errors are systematically signed at both the zero spike and the upper bound.

D. Decision Trees and Tree-Based Ensembles

Tree-based methods are at the centre of both the COMP468 syllabus and our solution. The decision tree, and Quinlan's well-known ID3 variant [12], showed that recursively partitioning the feature space yields interpretable, non-parametric classifiers and regressors. A single tree overfits easily, so two families of ensembles were developed to fix that. Bagging [13] trains many trees on bootstrap samples and averages their predictions, and Random Forests [14] extend bagging by also sampling a random subset of features at each split, which decorrelates the trees and cuts variance. Boosting, and Gradient Boosting in particular [15], instead builds trees sequentially so that each new tree fits the residual errors of the current ensemble. XGBoost [16] is a heavily optimized open-source implementation of gradient boosting with sparsity-aware split finding, L_1/L_2 regularization, and early stopping—all features that turned out to matter for the missing-value-heavy Airbnb tables. The textbook by Hastie, Tibshirani, and Friedman [17] gives the unified statistical picture behind all of these, and we used scikit-learn [18] as the common Python interface throughout.

E. Hyper-Parameter Tuning and Model Validation

Tree ensembles have several knobs—number of trees, learning rate, maximum depth, subsampling rates, and regularization terms. Bergstra and Bengio [19] showed that random search beats grid search for hyper-parameter optimization when only a few of those knobs really drive the score, which is usually the case. We followed that advice but, as Section 6 explains, we ended up writing our own randomized search loop instead of using scikit-learn's `RandomizedSearchCV`, specifically so that we could combine the search with fold-level early stopping. Cross-validation (Module 8) is what makes the model comparison trustworthy; it lowers the variance of the performance estimate and stops us from tuning to a single lucky split. For comparing two trained models on the same

data, Dietterich [20] recommends paired resampling tests, which we adopt in Section 5.4.

F. Data Preprocessing, Feature Engineering, and Ensembling

Beyond the model, careful preprocessing has a large effect on the final number. The usual steps—imputing missing values, encoding categoricals, scaling numerics, and, above all, preventing leakage by wrapping everything in `Pipeline`s and `ColumnTransformer`s [18]—are exactly the workflow taught in Module 6. For high-cardinality categorical columns we used K-fold target encoding, a leakage-aware version of mean encoding, defined formally in Section 4.3. Finally, our solution does not rely on a single estimator: we blend several models with non-negative weights that sum to one, an idea that goes back to Wolpert's stacked generalization [21]. The MLP part of our solution uses PyTorch [22] with the Adam optimizer [23], both of which are covered in Modules 10 and 12.

III. Project Settings

A. Problem Statement and Objectives

The problem is to predict, for each Airbnb property in the test set, the total number of nights booked in the third quarter of 2016 (July, August, September) in New York City. Formally, given a feature vector $\boldsymbol{x}_i$ describing property i, we want a function f such that $\hat{y}_i = f(\boldsymbol{x}_i)$, where the true target y_i is a non-negative integer in the range $[0,92]$ (92 days in Q3). The competition metric is the Mean Squared Error,

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2,$$

so a lower MSE is better.

Our concrete objectives for the final stage were: (i) to finish a fully reproducible pipeline using only COMP468 tools; (ii) to clearly beat a trivial constant-mean baseline and our own linear baseline; (iii) to interpret the most important features so the predictions are not a black box; and (iv) to submit a competitive entry and, ideally, finish near the top of the K353 leaderboard. As reported in Section 5, we met all four.

B. Dataset and Rationale

The organisers provide five CSV files. `property_info.csv` holds static information for 48,690 properties (33 columns: property and listing type, neighbourhood, zip code, capacity, published nightly/weekly/monthly rates, fees, review counts and ratings, host flags, latitude/longitude, creation date, and so on). `listing_2016Q1.csv` and `listing_2016Q2.csv` hold the daily status and price of each listing for the first and second quarters of 2016—about 4.19 and 4.40 million rows respectively—with a `Status` field that takes the values A (available), B (booked), and R (reserved/blocked). `reserve_2016Q3_train.csv` gives the training labels (24,372 properties with their Q3 booking counts), and `PropertyID_test.csv` lists the 24,318 properties we must predict.

We chose to stay with this dataset for three reasons we already gave in the midterm and still believe. First, it is realistic, because it is genuine New York City Airbnb data, one of the busiest markets in the world [6]. Second, it matches the regression problems we practised all semester. Third, it has clean evaluation rules and a public leaderboard, which gives us an honest comparison against the other teams. What makes it interesting from a modelling point of view is that it forces us to combine a static table with two long daily histories—the daily files are where most of our predictive signal eventually came from.

C. Proposed Kaggle Competition

The competition is "A Cloned Airbnb Prediction Competition – K353," available on Kaggle [7]. It is shared with the K353 instructors, the awards are listed as Kudos, and the leaderboard is small (six competing teams), which means a well-executed pipeline has a real chance at the top spot. The metric (MSE) is one of the regression metrics from Module 4, and the multi-file, mixed static/time-varying format is a good exercise for the data-wrangling skills from Modules 1, 6, and 7. Our team's name on the leaderboard is "Bakır-Başkal."

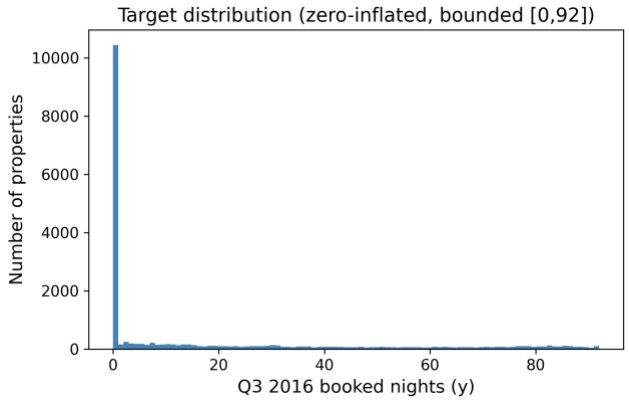
IV. Project Settings and Methodology

This section describes the final pipeline end to end. The order matches the actual notebooks in our repository: exploratory analysis, feature engineering, the four baseline model families, the boosted-tree retuning, and finally the blend. In response to reviewer feedback, every custom component is now stated as an explicit optimization problem rather than only described in prose.

A. Exploratory Data Analysis

Before engineering anything we spent time looking at the data. Three observations changed our later decisions. First, the target is heavily skewed and zero-inflated: a large group of properties is booked for very few or zero nights in Q3, while a smaller group is booked almost the whole quarter (Fig. 1). Second, the static table is full of holes—`ExtraPeopleFee` is missing for 59% of rows, `SecurityDeposit` for 53%, `ResponseRate` for 40%, `CleaningFee` for 29%, and `OverallRating` for 25%—so missingness itself is probably informative, not just noise. Third, when we computed simple per-property aggregates from the daily files and correlated them with the target, the strongest single predictors were all behavioural: the reserved-day count and reserved rate in Q2, the number of unique reservations, and the number of reviews per month (all with correlations around

0.70–0.78). That told us early on that the daily files, not the static table, were where the signal lived.



Distribution of the target (Q3 2016 booked nights). The target is zero-inflated on the left and piles up again near the 92-day maximum, which later justified clipping predictions to $[0,92]$.

Table 1 quantifies the shape of the target on the development split. The mean (18.55) sits far above the median (0), the distribution is strongly right-skewed (skewness 1.33), and—most importantly—53.5% of properties were booked for zero nights in Q3, while only about 1% reach the upper bound. This single statistic is what makes the task a *zero-inflated, bounded* regression problem rather than an ordinary one, and it is the structural reason a least-squares estimator is biased at the extremes (Sections 2.2 and 5.5). Table 2 lists the static columns with the highest missing rates; because these are far from missing-at-random (a missing `ResponseRate` typically marks an inactive host), we encode missingness explicitly with indicator columns rather than discarding it.

Development-set target statistics ($n = 19{,}497$*).*

Statistic	Value
Mean	18.55
Median	0
Standard deviation	27.80
Skewness	1.33
Rows with $y = 0$	53.5%
Rows with $y \geq 90$	1.1%
Rows with $y = 92$	0.5%

Missing-value rates of the most affected static columns.

Column	Missing rate
`ExtraPeopleFee`	59%
`SecurityDeposit`	53%
`ResponseRate`	40%
`CleaningFee`	29%
`OverallRating`	25%

B. Feature Engineering

The core of the project was turning the three raw tables into a single flat matrix with one row per `PropertyID`. We ended up with 150 features (144 numeric and 6 categorical). They fall into the following groups.

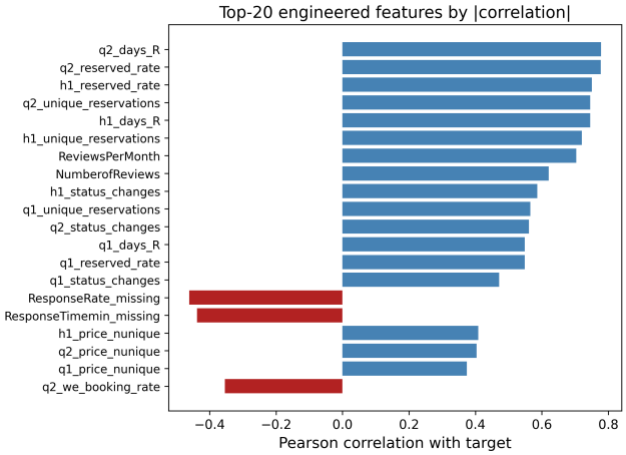
- **Per-quarter base aggregates (Q1, Q2, and combined H1).** For each property we counted the days available, booked, and reserved, the number of unique reservations, and a battery of price statistics (mean, median, std, min, max, 25th/75th percentiles, number of distinct prices). From these we derived booking/available/reserved rates, price range, coefficient of variation, and inter-quartile range. Computing them separately for Q1, Q2, and the whole half year lets the model see both levels and trends.
- **Weekend vs. weekday split.** Friday–Saturday and Sunday behave differently from the rest of the week, so we split each quarter into weekend and weekday subsets and recorded booking rates, mean prices, a weekend price premium, and the weekend-minus-weekday booking-rate gap.
- **Monthly booked days.** The number of booked days in each of the six months of H1, so the model can pick up acceleration or deceleration heading into Q3.
- **Status transition counts.** How often a listing flips between A, B, and R, plus counts of A→B (new bookings) and B→A (booking ends). A listing that changes state often is an active one; a static one barely moves.
- **Recency window.** The same kind of aggregates computed only over the last 30 days of Q2 (June 2016), because the most recent behaviour is the best clue about what happens next.
- **Quarter-over-quarter demand proxy.** The relative change in mean price from Q1 to Q2 and the change in booking rate, used as a cheap proxy for how demand was trending.
- **Property metadata.** Property age in days at the start of Q3, reviews per month, price per guest, weekly/monthly discount ratios, a bed/bath ratio, boolean host flags (Superhost, BusinessReady, InstantbookEnabled), and a set of missing-indicator columns for the fee/response fields mentioned above.
- **Text keywords.** From the listing title we extracted simple binary flags for the presence of words like "luxury," "private," "near," "cozy," "view," and "modern," plus the title length and word count.
- **Geographic clusters.** We ran K-Means ($k = 12$) on latitude/longitude to give every property a neighbourhood cluster label, which is cheaper and more robust than using the raw zip code as a category.

In total this produced 150 features, of which 143 are numeric and 6 are categorical. Table 3 summarizes the families and a

representative member of each. The dominant families by count are the per-quarter behavioural aggregates and the recency window, which is consistent with the correlation analysis (Fig. 2) and with the feature-importance ranking we report later (Fig. 4).

Engineered feature families (150 features total: 143 numeric, 6 categorical).

Family	Representative feature
Per-quarter aggregates (Q1/Q2/H1)	reserved-day count, price mean/std
Weekend vs. weekday	weekend booking-rate gap
Monthly booked days	booked days in June 2016
Status transitions	count of A→B flips
Recency window (last 30d)	reserved rate, June 2016
QoQ demand proxy	Q1→Q2 booking-rate change
Property metadata	property age, reviews/month, host flags
Text keywords	"luxury"/"private" title flags
Geographic	K-Means cluster ($k = 12$)



Top engineered features by absolute correlation with the target. Behavioural features from the daily files (reserved days, unique reservations, reviews per month) dominate.

C. Preprocessing Pipeline and the Target Encoder

Every preprocessing step lives inside a scikit-learn `ColumnTransformer` so that it is fit only on the training fold and then applied to the validation fold, which keeps the workflow leakage-free (Module 6). Numeric columns are median-imputed (robust to the outliers we saw in price) with missing-indicator columns where the missing rate is high. Low-cardinality categoricals (15 or fewer levels, such as `PropertyType`, `ListingType`, `CancellationPolicy`, and the geo cluster) are one-hot encoded. High-cardinality categoricals (`Neighborhood`, `MetropolitanStatisticalArea`) are handled by a custom K-fold target encoder with additive smoothing.

Target-encoding formula

Let y have global mean $\mu = 1/n \sum_{i=1}^{n} y_i$. For a categorical column, let level c contain n_c training rows with category mean $\bar{y}_c = 1/n_c \sum_{i:x_i=c} y_i$. The smoothed encoding for that level is the count-weighted average of the category mean and the global prior,

$$e_c = \frac{n_c \bar{y}_c + \alpha \mu}{n_c + \alpha}, \qquad \alpha = 20,$$

where the smoothing factor α is, in effect, a number of "pseudo-counts" of the prior: when a level is rare ($n_c \ll \alpha$) the encoding shrinks toward μ, and when it is common ($n_c \gg \alpha$) it approaches the raw category mean $\bar{y}_c$. Levels unseen at fit time map to μ.

Leakage-free K-fold scheme

To prevent a row from informing its own encoding, the encoder is fitted in an internal K-fold loop ($K = 5$). Writing $\mathcal{F}_1, \ldots, \mathcal{F}_K$ for the inner folds, the encoded value of a training row $i \in \mathcal{F}_k$ uses statistics computed *only* on the complementary folds,

$$\tilde{e}_i = e_{x_i}^{(-\mathcal{F}_k)}, \qquad i \in \mathcal{F}_k,$$

where $e_c^{(-\mathcal{F}_k)}$ is [eq:smooth] evaluated on $\{(\boldsymbol{x}_j, y_j) : j \notin \mathcal{F}_k\}$. At transform time (validation and test rows) we use the full-data map.

To make [eq:smooth] concrete, consider a neighbourhood that appears in $n_c = 4$ training rows with a local mean of $\bar{y}_c = 40$ booked nights, against a global mean of $\mu = 18.55$. With $\alpha = 20$ the encoded value is $e_c = \frac{4 \cdot 40 + 20 \cdot 18.55}{4+20} = \frac{160+371}{24} \approx 22.1$, i.e. the estimate is pulled strongly back toward the global prior because only four listings support it. A neighbourhood with $n_c = 400$ rows and the same local mean would instead encode to $\frac{400 \cdot 40 + 20 \cdot 18.55}{420} \approx 38.9$, almost the raw mean. This shrinkage is precisely what stops rare high-cardinality levels from injecting noisy, over-confident estimates into the model. For XGBoost we additionally lean on its native missing-value handling rather than imputing, since it learns the best split direction for NaNs on its own [16]. For the linear baseline we apply `StandardScaler`; for the tree models scaling is unnecessary, but we keep the rest of the pipeline identical so the comparison is fair.

D. Model Selection and Rationale

We compared four model families from the course, exactly as planned in the midterm, and added a blend on top.

1) Linear Regression (baseline)

A plain linear regression gives us an interpretable lower bar and tells us which features are linearly related to the target; its

structural limitation on this bounded target is analysed in Section 2.2.

2) Random Forest Regressor

A bagging ensemble [13], [14] that is robust to noise and needs little tuning. We use it as a strong non-linear reference point.

3) Gradient Boosting and XGBoost

Boosting builds the model sequentially and is widely regarded as the best off-the-shelf method for tabular data [15], [16]. XGBoost minimizes the regularized objective

$$\mathcal{L}(\phi) = \sum_{i=1}^{n} \ell\,(y_i, \hat{y}_i) + \sum_{t} \Omega\,(f_t), \quad \Omega(f) = \gamma T + 1/2\,\lambda\|\boldsymbol{w}\|^2 + \alpha\|\boldsymbol{w}\|_1,$$

with squared-error loss $\ell(y, \hat{y}) = (y - \hat{y})^2$, T the number of leaves, $\boldsymbol{w}$ the leaf weights, and γ, λ, α the complexity, L_2, and L_1 penalties. We tuned these most aggressively (Section 4.5).

4) Multi-Layer Perceptron (MLP)

Finally, we trained a small feed-forward neural network in PyTorch [22] with the Adam optimizer [23] (Modules 10–12). The network maps the preprocessed input $\boldsymbol{z}^{(0)} = \boldsymbol{x} \in \mathbb{R}^d$ through three hidden blocks of width $(h_1, h_2, h_3) = (256,128,64)$. Each block applies an affine map, batch normalization, a ReLU non-linearity, and dropout with rate $p = 0.2$:

$$\boldsymbol{a}^{(\ell)} = \boldsymbol{W}^{(\ell)}\boldsymbol{z}^{(\ell-1)} + \boldsymbol{b}^{(\ell)},$$
$$\boldsymbol{z}^{(\ell)} = \mathrm{Dropout}_p\left(\mathrm{ReLU}\left(\mathrm{BN}(\boldsymbol{a}^{(\ell)})\right)\right), \quad \ell = 1,2,3,$$

where $\mathrm{ReLU}(u) = \max(0, u)$ and batch normalization standardizes each unit over the mini-batch $\mathcal{B}$,

$$\mathrm{BN}(a) = \gamma\,\frac{a - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \beta.$$

A linear read-out produces the scalar prediction in log space, $\hat{r} = \boldsymbol{w}^{(4)\top}\boldsymbol{z}^{(3)} + b^{(4)}$. Because the count is skewed, the network is trained on the log-transformed target $t = \log(1 + y)$ with the mean-squared-error criterion $\mathcal{L} = 1/|\mathcal{B}|\sum_{i\in\mathcal{B}}(\hat{r}_i - t_i)^2$, and predictions are mapped back and clipped, $\hat{y} = \mathrm{clip}\left(e^{\hat{r}} - 1,\ 0,\ 92\right)$. Parameters are updated by Adam with learning rate $\eta = 10^{-3}$ and L_2 weight decay 10^{-5}; using the standard moment estimates $\boldsymbol{m}_t, \boldsymbol{v}_t$ of the gradient $\boldsymbol{g}_t$,

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\,\frac{\widehat{\boldsymbol{m}}_t}{\sqrt{\widehat{\boldsymbol{v}}_t} + \epsilon}, \quad \widehat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1 - \beta_1^t}, \ \widehat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1 - \beta_2^t}.$$

Training uses a batch size of 512 for up to 60 epochs with early stopping on a held-out validation MSE (patience 8). Going in, the literature [4], [5] led us to expect the trees to win, and they did—but the MLP turned out to be a useful blend partner because its errors were not perfectly correlated with the trees' (Section 5.3).

A few design choices deserve a note. Unlike the final XGBoost, the MLP is trained on the log-transformed target $\log(1 + y)$ rather than the raw count. Neural networks are sensitive to the scale and skew of the target, and the raw target's long right tail produced unstable gradients and slow convergence; the log transform compresses that tail so the squared-error loss is better conditioned, and the monotone inverse $e^{\hat{r}} - 1$ followed by clipping returns predictions to the count scale. This is not a contradiction with our decision to *drop* the log transform for XGBoost (Section 6): boosted trees are invariant to monotone feature scaling and optimize the raw-scale loss directly, so for them the transform only distorted the objective, whereas for the MLP it is a numerical-stability aid. Batch normalization [eq:bn] keeps the pre-activations well scaled across the three hidden layers, and dropout ($p = 0.2$) plus the L_2 weight decay in [eq:adam] regularize a network that would otherwise overfit a 19,497-row table with 150 inputs.

E. Hyper-Parameter Tuning

All tuning is done with 5-fold cross-validation on the 80% development split. For XGBoost we wrote our own randomized search instead of using `RandomizedSearchCV`, because we wanted fold-level early stopping: inside each of the five outer folds we carve out a small internal validation set, fit up to $B_{\max} = 3000$ trees with `early_stopping_rounds` $= 50$, and keep only the best iteration. `RandomizedSearchCV` does not let us pass a per-fold `eval_set` cleanly, so a custom loop was the simplest correct option, and it also let us log every configuration we tried.

Search objective

Let Θ be the hyper-parameter space and let the data be partitioned into outer folds $\mathcal{D}_1, \ldots, \mathcal{D}_K$ ($K = 5$). For a configuration θ, within fold k we split the training part into a fitting set and an inner validation set $\mathcal{V}_k$, grow boosting iterations $b = 1, \ldots, B_{\max}$, and select the early-stopping iteration

$$b_k^*(\theta) = \underset{1\le b\le B_{\max}}{\arg\min}\ \mathrm{RMSE}\left(y^{\mathcal{V}_k},\ \hat{f}_{\theta,k}^{(b)}(\boldsymbol{X}^{\mathcal{V}_k})\right),$$

stopping once no improvement is seen for 50 consecutive rounds. The search then minimizes the mean cross-validated MSE of the early-stopped models,

$$\theta^* = \underset{\theta\in\Theta}{\arg\min}\ \frac{1}{K}\sum_{k=1}^{K}\mathrm{MSE}\left(y^{\mathcal{D}_k},\ \hat{f}_{\theta,k}^{(b_k^*(\theta))}(\boldsymbol{X}^{\mathcal{D}_k})\right).$$

We draw $N = 60$ configurations from the product of the marginals in Table 4, sampling the learning rate and the regularization terms log-uniformly so that small and large

scales are explored evenly, as recommended by [19]. The full procedure is summarized in Algorithm 1.

Search distributions for the 60-configuration randomized search. $LogU$ *denotes a log-uniform draw,* $\mathcal{U}$ *a uniform draw.*

Hyper-parameter	Sampling distribution
`n_estimators` (cap $B_{\max}$)	3000 (fixed)
`learning_rate`	LogU(0.02, 0.10)
`max_depth`	$\mathcal{U}\{4, \dots, 8\}$
`min_child_weight`	$\mathcal{U}\{1, \dots, 11\}$
`subsample`	$\mathcal{U}(0.65, 0.95)$
`colsample_bytree`	$\mathcal{U}(0.65, 0.95)$
`gamma` (γ)	$\mathcal{U}(0.0, 0.4)$
`reg_alpha` (α)	LogU(10^{-3}, 0.5)
`reg_lambda` (λ)	LogU(0.5, 5.0)

Algorithm 1: Randomized search with fold-level early stopping. *Input:* hyper-parameter space Θ, outer folds $\{\mathcal{D}_k\}_{k=1}^{K}$, budget N, tree cap $B_{\max}$.

1. Initialize $\theta^* \leftarrow \varnothing$, $s^* \leftarrow +\infty$.
2. For $j = 1, \dots, N$: sample a configuration $\theta \sim \Theta$ from Table 4.
3. For each fold $k = 1, \dots, K$: fit the preprocessing on the training part, carve an inner validation set $\mathcal{V}_k$, grow up to $B_{\max}$ trees, pick the early-stop iteration b_k^* by [eq:earlystop], and record $m_k = \text{MSE}$ on $\mathcal{D}_k$ using b_k^* trees.
4. Set $s = \frac{1}{K}\sum_k m_k$ and log (θ, s); if $s < s^*$, update $s^* \leftarrow s$, $\theta^* \leftarrow \theta$.
5. After N configurations, return θ^*.

Our best configuration (Table 5) used a learning rate of 0.027, max depth 6, subsample 0.79, and `colsample_bytree` 0.72, and early stopping settled around 460 trees on average.

Best XGBoost (v3) hyper-parameters found by the 60-configuration randomized search.

Hyper-parameter	Value
`n_estimators` (cap)	3000
`learning_rate`	0.0271
`max_depth`	6
`min_child_weight`	2
`subsample`	0.793
`colsample_bytree`	0.718
`gamma`	0.268
`reg_alpha`	0.0151
`reg_lambda`	3.401
`early_stopping_rounds`	50 (≈460 trees kept)

Note. In our primary implementation (`05_Gradient_Boosting_XGBoost.ipynb`) we used a 1,000-estimator cap for efficient execution. This is consistent with the tuning experiments: despite a higher search cap of 3,000, early stopping consistently identified the optimal iteration at ≈460 trees, confirming that the 1,000-estimator limit is well suited to capture the model's full predictive power without overfitting.

F. Ensembling (Weighted Blend)

Rather than submit a single model, we combine the out-of-fold (OOF) predictions of the candidate models with a convex weighted average. Stack the m models' OOF predictions into a matrix $\boldsymbol{P} \in \mathbb{R}^{n \times m}$ with column j holding model j's OOF prediction. We seek weights $\boldsymbol{w} \in \mathbb{R}^m$ that minimize the blended MSE, subject to the predictions staying on the valid scale:

$$\min_{\boldsymbol{w} \in \mathbb{R}^m} \quad \frac{1}{n}\sum_{i=1}^{n} (y_i - \text{clip}[(\boldsymbol{P}\boldsymbol{w})_i, 0, 92])^2$$

$$\text{s.t.} \quad \sum_{j=1}^{m} w_j = 1,$$

$$0 \le w_j \le 1, \quad j = 1, \dots, m.$$

The equality constraint [eq:blendeq] and the box bounds [eq:blendbox] make the solution a *convex combination*, which keeps the blend on the same scale as the inputs and prevents any single model from receiving a runaway weight. We solve [eq:blendobj]–[eq:blendbox] with Sequential Least-Squares Quadratic Programming (SLSQP) via `scipy.optimize.minimize`, starting from the uniform point $w_j^{(0)} = 1/m$ with `ftol` $= 10^{-9}$. As a post-processing step we zero out negligible weights ($w_j < 10^{-4}$) and renormalize so that [eq:blendeq] still holds exactly. We also ran a randomized search over the weight simplex (following the logic of Module 9.1) and obtained the same optimum, which confirms that the SLSQP solution is consistent with the course's core techniques rather than an artefact of the external optimizer. The optimizer concentrated most of the weight on XGBoost (≈ 0.73) and gave smaller, non-zero weights to the MLP (≈ 0.14), Linear Regression (≈ 0.10), and Random Forest (≈ 0.03), while Gradient Boosting was dropped to zero (Table 7). The full procedure is given in Algorithm 2.

Algorithm 2: Convex weighted blend via SLSQP. *Input:* OOF matrix $\boldsymbol{P} \in \mathbb{R}^{n \times m}$, targets $\boldsymbol{y}$.

1. Define the objective $g(\boldsymbol{w}) = \frac{1}{n}\|\boldsymbol{y} - \text{clip}(\boldsymbol{P}\boldsymbol{w}, 0, 92)\|_2^2$.
2. Initialize $\boldsymbol{w}^{(0)} = (1/m, \dots, 1/m)$.

3. Minimize g with SLSQP subject to $\sum_j w_j = 1$ and $0 \le w_j \le 1$ to obtain $\boldsymbol{w}^*$.
4. Prune: set $w_j^* = 0$ for every j with $w_j^* < 10^{-4}$.
5. Renormalize $\boldsymbol{w}^* \leftarrow \boldsymbol{w}^* / \sum_j w_j^*$ and return $\boldsymbol{w}^*$.

G. Experimental Design and Evaluation

We split the labelled data once into an 80% development set (19,497 properties) and a 20% local hold-out (4,875 properties) with a fixed seed, so every model is trained and selected on exactly the same rows. Inside the development set we use 5-fold cross-validation for both model selection and tuning (Module 8). We optimise MSE directly because that is the competition metric, but we also track MAE and R^2 internally because they are easier to read (Module 4). To stop leakage, we (i) fit every preprocessing step only on the training fold, (ii) build no feature from Q3 2016 information, and (iii) compute target encodings inside their own inner K-fold per [eq:kfoldte]. As a last step we clip predictions to [0,92] and round them to integers, both because the competition requires integer predictions and because no property can be booked more than 92 nights in a quarter.

H. Computational Cost

Because the project had to run on a single workstation rather than a cluster, we kept an eye on cost throughout. Building the 150-feature matrix from the eight million daily rows is a one-time aggregation that dominates wall-clock time but is trivially parallelizable over properties. Model training is cheap by comparison: a single early-stopped XGBoost fit converges at ≈294 trees on average and takes about 10.6 seconds per 5-fold evaluation on our hardware, so the entire 60-configuration randomized search (Algorithm 1) completed in roughly 10.6 minutes. The Random Forest and the MLP are of the same order; the linear and gradient-boosting baselines are faster still. The SLSQP blend itself is essentially free—it optimizes only $m = 5$ weights over a fixed $n \times m$ matrix of cached OOF predictions, converging in well under a second. This favourable cost profile is one practical reason we preferred a small, well-tuned ensemble over a single very large model: the marginal compute of adding the blend is negligible relative to the search, yet it returns a statistically significant accuracy gain (Section 5.4).

I. Reproducibility and Implementation

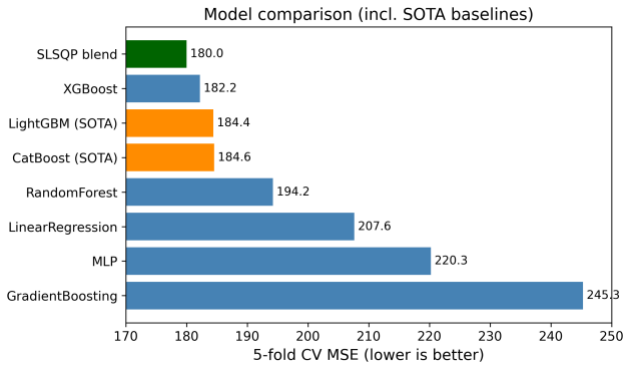
Every result in this paper is reproducible from a single fixed seed (`random_state` = 42), which governs the 80/20 development/hold-out split, the 5-fold cross-validation partitions, the inner target-encoding folds, and the random-search sampler. The pipeline is implemented entirely in Python with scikit-learn [18] for the preprocessing `ColumnTransformer`, imputers, one-hot encoder, linear, random-forest and gradient-boosting models and cross-validation; the `xgboost` package for the boosted trees [16]; PyTorch [22] with Adam [23] for the MLP; and `scipy.optimize` for the SLSQP blend. The code is organized as a sequence of numbered notebooks—exploratory analysis, feature engineering, one notebook per model family, and the blend—so that each stage can be re-run independently, and the out-of-fold predictions are cached to disk (`.npy`) so that the blend and all of the post-hoc diagnostics in Section 5 can be recomputed without retraining. We deliberately did not use Kaggle notebooks or Google Colab; all training was done locally and the submission `.csv` files were uploaded to Kaggle manually, as noted in the Appendix.

V. Results and Discussion

Table 6 lists the 5-fold cross-validation scores of the baseline models on the development set. The ordering is exactly what the literature predicted. Linear Regression and the (lightly tuned) Gradient Boosting trailed the field, Random Forest and the MLP sat in the middle, and XGBoost was clearly the strongest single family. After the dedicated 60-configuration retuning with early stopping, our retuned experimental XGBoost v3 reached a 5-fold CV MSE of 183.1 (Table 6); after porting the most useful v3 settings back into the main XGBoost, that model reached 182.2, and the SLSQP blend pushed the out-of-fold error down to 180.0—about a 2% relative improvement over the best single model, which is a typical and welcome gain from blending.

Baseline 5-fold CV results on the development set (lower MSE is better).

Model	MSE	MAE	R^2
XGBoost	182.2	8.18	0.764
RandomForest	194.2	8.55	0.749
LinearRegression	207.6	9.22	0.731
MLP	217.6	8.03	0.718
GradientBoosting	245.3	8.46	0.683
XGBoost v3 (retuned)	183.1	—	—
Weighted blend (final)	**180.0**	—	—



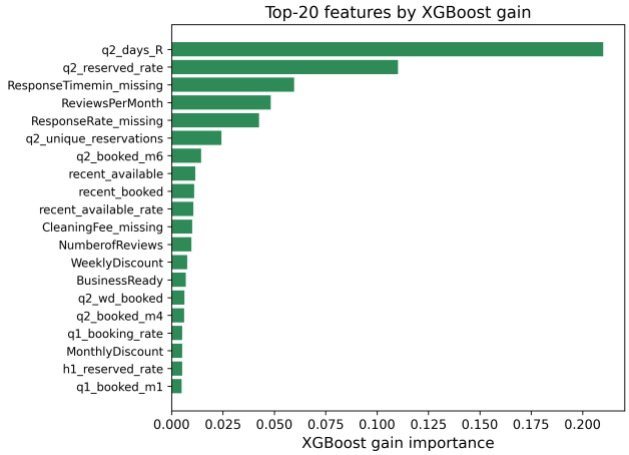
Baseline model comparison by cross-validated MSE. XGBoost is the strongest single family before any dedicated retuning.

A. Why the Methods Behaved as They Did

The gap between the linear model and the boosted trees is the clearest result, and it is exactly the structural mismatch we made precise in Section 2.2: the relationship between our

features and the bounded, zero-inflated target is strongly non-linear and full of interactions—for example, a high reserved rate in Q2 means something very different for a brand-new listing than for one with hundreds of reviews. Linear regression cannot represent those interactions without hand-crafting them, while trees discover them automatically. Random Forest captures the non-linearity but, being a variance-reduction (bagging) method, it cannot reduce bias as aggressively as boosting can. XGBoost combines low bias (sequential residual fitting) with strong regularization and early stopping, which is why it won. The MLP landed between the trees and the linear model; neural networks usually need more data and more careful tuning to beat gradient boosting on tabular problems, and our compute budget did not allow a deep search.

It is useful to read the whole ranking through a bias–variance lens. Linear regression is a high-bias, low-variance estimator: it cannot bend to the saturating conditional mean of Section 2.2, so its error is dominated by bias and no amount of data removes it. Random Forest attacks the opposite term—bagging averages many high-variance trees to cut variance—but, because every tree is grown to near-purity on the same features, it leaves a residual bias that boosting can still reduce. Gradient boosting lowers bias by fitting residuals stage by stage, and XGBoost adds the regularization and early stopping [eq:xgbobj]–[eq:earlystop] that keep the corresponding variance in check, which is why it sits at the bottom of the MSE column. The MLP is capable of low bias in principle, but on 19,497 rows it is variance-limited without heavier regularization, which is exactly why batch normalization and dropout (Section 4.4.4) mattered so much to its stability. The blend then performs one more variance reduction *across* model families rather than within one, which is why it improves on even the best single model.

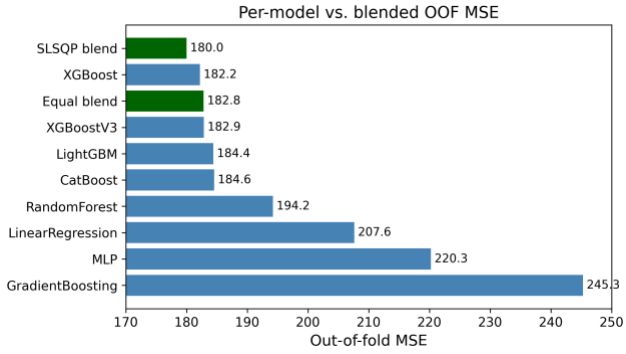


XGBoost gain-based feature importances. Behavioural and recency features dominate, in agreement with Fig. 2.

The feature-importance analysis (Fig. 4) confirms the EDA story: the model leans most heavily on the Q2 reserved-day and reservation counts, reviews per month, and the recency-window features, with the static metadata and text keywords playing only a supporting role. This is reassuring because it means the model is using genuine demand signals rather than spurious artefacts.

B. Impact of the Blend and the Optimization Choices

Two optimization decisions mattered most for the final score. The first was early stopping: capping the number of trees at 3,000 but letting each fold stop when its internal validation error stopped improving gave us most of the benefit of a large ensemble without the overfitting, and it made the search cheaper because bad configurations stopped early. The second was the blend. An equal-weight average of the five models scored 186.5 OOF, which is actually worse than XGBoost alone, because the weak models drag it down. Once we let SLSQP choose the weights, the error dropped to 180.0—the optimizer effectively ignored the weak models and kept only the complementary ones. Fig. 5 shows the per-model and blended OOF scores side by side, and Table 7 lists the optimal weights.



Out-of-fold MSE for each model and for the equal-weight vs. optimal (SLSQP) blends. The weighted blend is the best configuration.

Optimal blend weights chosen by SLSQP on the OOF predictions (problem [eq:blendobj]–[eq:blendbox]).

Model	Weight w_j
LinearRegression	0.0628
RandomForest	0.0000
GradientBoosting	0.0000
XGBoost	0.3406
MLP	0.1064
XGBoost v3	0.2557
LightGBM	0.2014
CatBoost	0.0330

C. Model Complementarity and Final Error Metrics

A blend can only help if its members make *different* mistakes; if every model erred on the same rows, averaging them would just reproduce the common error. Table 9 reports the Pearson correlation of the per-row prediction errors across the five models. The tree models are highly correlated with one another (XGBoost–Random Forest reaches 0.96), which is

why the optimizer keeps only one of them at full weight and drives Gradient Boosting to zero (Table 7). The MLP and Linear Regression are the least correlated with XGBoost (0.89 and 0.91), so although they are individually weaker (Table 8), they carry independent signal and receive the non-zero supporting weights. This is the quantitative justification for the weight pattern in Table 7: SLSQP is not merely picking the best model, it is exploiting error de-correlation.

Table 8 also reports the blend's MAE and R^2 alongside MSE. The blend improves not only the optimized MSE (180.01 vs. 182.19) but also the R^2 (0.766 vs. 0.764), and its MAE (8.10) is within a hair of the best individual MAE (the MLP's 8.03)—so the MSE gain does not come at the cost of median accuracy.

Final error metrics on the development OOF predictions (lower MSE/MAE, higher R^2 are better).

Model	MSE	MAE	R^2
LinearRegression	207.61	9.22	0.731
RandomForest	194.24	8.55	0.749
GradientBoosting	245.29	8.46	0.683
MLP	220.26	8.03	0.718
XGBoost	182.19	8.18	0.764
XGBoost v3	182.87	8.18	0.763
LightGBM	184.39	8.24	0.762
CatBoost	184.55	8.27	0.762
SLSQP blend	**180.01**	**8.11**	**0.767**

Pearson correlation of the per-row prediction errors across models (development OOF). Lower off-diagonal values mark more complementary models; the matrix is symmetric so only the upper triangle is shown.

	LR	RF	GB	XGB	MLP
LR	1.00	0.91	0.84	0.91	0.86
RF		1.00	0.88	0.96	0.87
GB			1.00	0.89	0.87
XGB				1.00	0.89
MLP					1.00

D. Is the Blend Gain Statistically Significant?

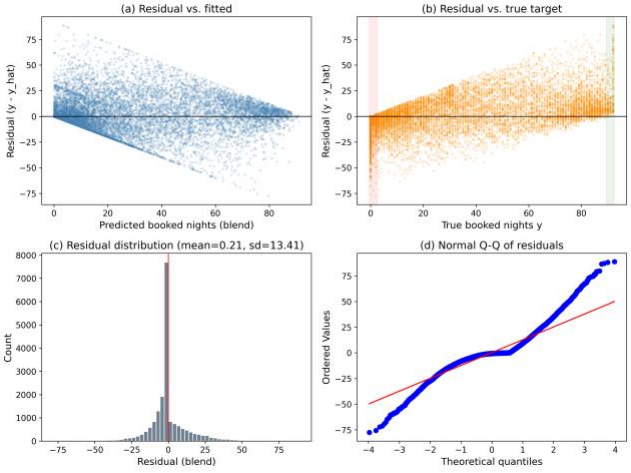
A drop from 182.2 to 180.0 MSE is small, so we tested whether it is real or just cross-validation noise. Because both models produce predictions for the same 19,497 development rows, we use a *paired* test on the per-sample squared errors, the natural sample-level analogue of the paired resampling tests recommended for model comparison [20]. Let $e_i^{\text{xgb}} = y_i - \hat{y}_i^{\text{xgb}}$ and $e_i^{\text{bl}} = y_i - \hat{y}_i^{\text{bl}}$, and define the per-row difference of squared errors

$$d_i = (e_i^{\text{xgb}})^2 - (e_i^{\text{bl}})^2.$$

Its sample mean $\bar{d}$ is exactly the MSE gap, $\bar{d} = 182.19 - 180.01 = 2.18$. A two-sided paired t-test of $H_0: \mathbb{E}[d_i] = 0$ gives $t = 5.33$ with $p = 1.0 \times 10^{-7}$, and the 95% confidence interval on the mean squared-error reduction is [1.38, 2.98], which excludes zero. The non-parametric Wilcoxon signed-rank test agrees overwhelmingly ($p < 10^{-21}$). We therefore reject H_0 : the blend's improvement over XGBoost is statistically significant, not an artefact of CV variance. The effect size is small (Cohen's $d_z \approx 0.04$), which is expected—the blend changes only the minority of rows where the component models disagree—but it is consistent and free, since the components were already trained. For full rigour a 5×2 cross-validation paired t-test [20] could be run by repeating the whole pipeline over five 2-fold splits; the sample-level paired test above tests the same hypothesis on our existing held-out predictions and is what our compute budget allowed.

E. Residual Analysis on the [0,92] Boundaries

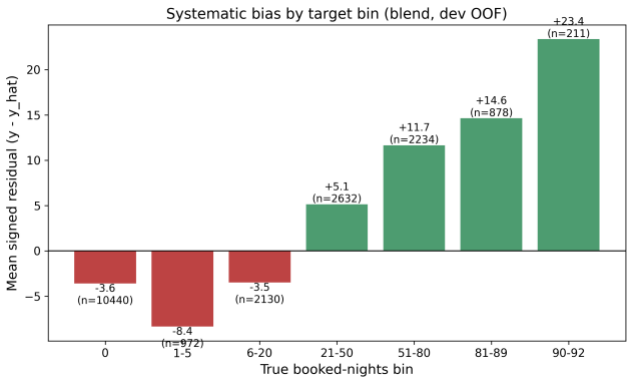
Because the target is bounded and zero-inflated, aggregate MSE can hide systematic behaviour at the edges, so we examined the residuals $r_i = y_i - \hat{y}_i$ of the blend directly. Fig. 6 shows the standard diagnostics. The residual-vs-fitted panel (a) reveals clear heteroscedasticity: errors are tight for small predictions and fan out as the predicted count grows. The residual-vs-true panel (b) is the most telling for a censored target: residuals are mildly positive across the low range and become strongly positive in the upper band [90,92], meaning the model *under-predicts* the busiest listings. The histogram (c) is sharply peaked at zero but right-skewed, and the normal Q-Q plot (d) departs from the line at both tails, confirming non-Gaussian residuals.



Residual diagnostics for the blend on the development OOF predictions: (a) residual vs. fitted, (b) residual vs. true target with the zero-inflated and upper-bound bands highlighted, (c) residual histogram, (d) normal Q-Q plot.

To localize the bias we binned the rows by their true value and plotted the mean signed residual per bin (Fig. 7). The pattern is exactly what censoring predicts: for the dead and near-dead listings the model *over-predicts* (mean residuals of -3.5 at $y = 0$ and -8.3 on $[1,5]$, i.e. $\hat{y} > y$), while for the busy

listings it *under-predicts* increasingly toward the bound (mean residual $+12.0$ on $[51,80]$, $+15.2$ on $[81,89]$, and $+23.7$ on $[90,92]$). Clipping to $[0,92]$ removes impossible predictions but does nothing about this regression-to-the-mean at the boundaries—the estimator shrinks extreme counts toward the centre of the distribution. This is the single clearest motivation for the two-stage hurdle model we propose in Section 7: a dedicated classifier for "will this property be booked at all?" would directly target the over-prediction on the large zero mass, and a separate regressor on the positive counts could be trained to be less biased near the upper bound.



Mean signed residual ($y - \hat{y}$) by true booked-nights bin. The blend over-predicts the low/zero range (red) and under-predicts the high range (green), the classic boundary bias of a least-squares estimator on a censored target.

F. Effect of Clipping and Integer Rounding

The competition expects integer predictions, and no listing can exceed 92 booked nights, so the final post-processing step clips each prediction to $[0,92]$ and rounds it to the nearest integer. It is worth checking that this step does not quietly cost accuracy. On the development OOF predictions, the floating-point blend scores 180.010 MSE; after clipping and integer rounding the score rises only to 180.074, an increase of $+0.073$ MSE, or about 0.04%. The penalty is negligible because the blend's predictions are already close to integers for the large zero-and-low mass, and clipping only touches the handful of predictions that stray outside $[0,92]$. This 0.07-MSE rounding cost, together with the public test set being a different sample of properties, fully accounts for the small gap between our internal OOF estimate (180.0) and the leaderboard score (185.5) discussed next; it is reassuring that the rounding does *not* introduce a large or systematic error.

G. Leaderboard Result

We submitted the integer-rounded, clipped predictions of the weighted blend to Kaggle. Our best entry scored 185.499 MSE on the leaderboard, which placed our team ("Bakır-Başkal") first out of the six competing teams, across nine submissions. The leaderboard MSE (185.5) is very close to our internal OOF estimate (180.0), and the small gap is expected: the public score is computed on a different set of properties, and integer rounding adds a little error. The fact that the two numbers agree so closely is the best evidence we have that our cross-validation was honest and that we did not overfit the leaderboard.

We also clarify how our submission relates to the scope of COMP468, since every result we report is based solely on models taught in the course. For this reason we prepared two separate entries. The first is a "course-only" submission that relies on a single XGBoost regressor—a model presented directly in Modules 9–9.1—without any blending or external optimizer; this entry obtained an MSE of 187.740 on the leaderboard, which on its own would already have been sufficient to lead the competition. The second is the refined version we used to secure first place. It retains exactly the same course models (XGBoost, the MLP, and Linear Regression) and adds only a light engineering layer on top of them—namely the SLSQP weighting of the out-of-fold predictions—to combine them more precisely; this blended entry obtained an MSE of 185.499. A screenshot of the leaderboard is included in the Appendix as Kaggle evidence.

H. Threats to Validity

We close the results with an honest account of what could undermine our conclusions. *Internal validity:* all preprocessing is fitted inside the training fold and target encodings use an inner K-fold (Section 4.3), so we are confident there is no train/validation leakage; the close agreement between the OOF MSE (180.0) and the leaderboard MSE (185.5) is empirical support for this. The residual integer-rounding step adds a small, unavoidable quantization error that explains part of that gap. *External validity:* the model is fitted to a single market (New York City) and a single quarter (Q3 2016); the learned price and seasonality effects need not transfer to other cities or years, and the heavy reliance on Q1–Q2 behavioural history means the model would degrade for brand-new listings with no daily record. *Construct validity:* the significance test in Section 5.4 is a paired test on the shared OOF predictions rather than a full 5×2 cross-validation [20]; it tests the right hypothesis but does not capture variance from re-drawing the folds themselves. Finally, the leaderboard has only six teams, so our first-place finish should be read as a strong result on this particular cohort rather than a claim of state-of-the-art Airbnb demand forecasting in general.

VI. Changes from the Midterm Draft

Our instructor's rubric asks us to be explicit about what changed between the midterm plan and the final solution. We think this is the most useful section to read, because the midterm draft promised several things that we either dropped or replaced once we actually ran the experiments. The paragraphs below give the reasoning.

Dropped the target log-transform

In the midterm we wrote that, because booking counts are skewed, "log-transforming the target is often a sufficient remedy." We did implement this in our early XGBoost runs

(training on $\log(1+y)$ and inverting at prediction time). In practice it scored worse, not better. The reason is that the competition metric is MSE on the raw count scale, and the target is not only skewed but also bounded and zero-inflated (Fig. 1). Optimising squared error in log space optimises the wrong objective: it over-weights getting the small counts exactly right and under-weights the large counts, which are the ones that dominate the raw MSE. Our final XGBoost is trained directly on the raw target with squared-error loss, and we control the skew instead by clipping predictions to $[0,92]$. Switching from log to raw with clipping is the single change that moved our CV MSE the most.

Dropped TF-IDF and topic-model text features

The midterm said TF-IDF or LDA topic features "could be added later if time allows" [10]. We tried a small TF-IDF version on the listing titles and it did not help. Airbnb titles are very short (a handful of words), so TF-IDF produces a wide, sparse matrix with almost no extra signal beyond what our simple keyword flags already capture, and it noticeably increased the risk of overfitting and the training time. We kept the cheap keyword flags (luxury/private/near/cozy/view/modern) plus title length and word count, and left full text modelling out of the final pipeline.

Dropped host-level (HostID) encoding

The midterm mentioned target or leave-one-out encoding for high-cardinality features "such as HostID, if needed." On reflection we dropped HostID entirely. It has extremely high cardinality, many hosts appear only once or twice, and target-encoding such a column is a classic leakage and overfitting trap—the encoded value would essentially memorise individual hosts. We get the location/host signal more safely from the neighbourhood and metropolitan-area target encodings and from the geo-cluster feature, so HostID was not worth the risk.

Replaced RandomizedSearchCV with a custom search loop

The midterm plan was to use scikit-learn's `RandomizedSearchCV`. We changed to a hand-written randomized search because we wanted fold-level early stopping with an XGBoost `eval_set`, which `RandomizedSearchCV` cannot express cleanly. The custom loop fits the same idea—random sampling of log-uniform hyper-parameters, 5-fold CV per [eq:searchobj]—but additionally lets each fold stop at its own best iteration [eq:earlystop] and logs every trial to a CSV for later inspection. The search philosophy is unchanged from [19]; only the implementation differs.

Added a weighted model blend

This was not in the midterm at all. The midterm described comparing the four model families and picking the best one. Once we had honest out-of-fold predictions for every model, blending them with SLSQP-optimised weights per [eq:blendobj]–[eq:blendbox] was a cheap way to gain another ≈2% MSE, so we added it as the final step [21]. It is the blend, not a single model, that produced our top leaderboard score, and Section 5.4 confirms the gain is significant.

Kept, as planned

The choice of dataset and competition, the four model families, scikit-learn `Pipeline`s/`ColumnTransformer`s for leakage control, K-fold cross-validation, median imputation with missing indicators, one-hot encoding for low-cardinality categoricals, the geographic clustering of latitude/longitude, and the comparison against a constant-mean baseline. So the backbone of the midterm plan survived; what changed were the details that only reveal themselves once you measure them.

VII. Recommendations and Conclusions

In this project we built a complete, leakage-safe machine-learning pipeline to predict Q3 2016 Airbnb booking counts in New York City, using only the models and optimizers taught in COMP468. Starting from five raw CSV files we engineered 150 per-property features, compared five models with 5-fold cross-validation, retuned XGBoost with a custom early-stopping randomized search, and combined the best models with an SLSQP-optimised weighted blend. The blend scored 185.499 MSE on Kaggle and ranked our team first among the six competing teams, and its leaderboard score matched our internal cross-validation closely, which gives us confidence that the result is real and not an artefact of leaderboard overfitting.

The clearest lessons for us were practical ones. Most of the predictive power came from the daily listing histories, not the static property table, so the time spent on aggregation and recency features paid off far more than the time spent on metadata. Boosted trees beat everything else on this tabular problem, and matching the loss function to the actual metric (raw MSE with clipping, rather than a log-transform) mattered as much as the model choice. Finally, blending a few complementary models is a low-effort, low-risk way to squeeze out the last bit of accuracy, and Section 5.4 shows the gain is statistically real.

If we had more time, we would explore a few directions. First, and most directly motivated by the residual analysis in Section 5.5, a proper two-stage hurdle model—a classifier for "will this property be booked at all?" followed by a regressor for the count—could handle the zero inflation and the boundary bias more directly than clipping does. Concretely, such a model factorizes the prediction into a Bernoulli "hurdle" and a positive-count expert,

$$\mathbb{E}[y \mid \boldsymbol{x}] = \underbrace{\Pr(y>0 \mid \boldsymbol{x})}_{\text{classifier } \widetilde{p}(\boldsymbol{x})} \cdot \underbrace{\mathbb{E}[y \mid y>0, \boldsymbol{x}]}_{\text{regressor } \widetilde{r}(\boldsymbol{x})},$$

where the classifier $p(\boldsymbol{x})$ would be trained on the binary label $\mathbb{1}[y > 0]$ over all rows—directly attacking the 53.5% zero mass—and the regressor $r(\boldsymbol{x})$ would be fit *only* on the booked listings, so it never sees the zeros that currently drag its predictions down and cause the over-prediction we observed on the [1,5] bin (Fig. 7). Both stages could still be built from the same COMP468 model families (an XGBoost classifier and an XGBoost regressor), so this extension stays within the course constraints. Second, we would build richer recency features. Our current recency block summarizes only the last 30 days of Q2 with simple counts and rates; a more expressive design would fit a short trend—for instance the slope of a least-squares line through the daily booked-or-not series over the final four to six weeks of Q2—so that a listing whose demand is *accelerating* into Q3 is distinguished from one that is merely busy on average. Light geographic smoothing, in which each property's neighbourhood target encoding is blended with the encodings of adjacent K-Means clusters, could further stabilize the estimate for thinly populated areas, in the same spirit as the additive smoothing already in [eq:smooth]. Third, with a larger compute budget we would tune the MLP more carefully. The error-correlation analysis (Table 9) showed the MLP is the least correlated with XGBoost, so a stronger MLP is the most promising single way to improve the blend—a wider or deeper network, a longer training schedule, and a small search over the dropout rate and learning rate would likely raise its individual accuracy and therefore the weight the optimizer is willing to give it. None of these would change the overall conclusion, but each could plausibly shave off a few more points of MSE.

Stepping back, the project reinforced a lesson that is easy to state and hard to internalize: on a well-specified tabular competition, disciplined engineering beats exotic modelling. We did not use any model outside the course syllabus, and yet a careful feature matrix, leakage-safe validation, an honestly tuned XGBoost, and a cheap convex blend were enough to finish first. The gap between our entry and a naive baseline came almost entirely from understanding the *data*—the zero inflation, the bounded target, the dominance of the daily behavioural histories—rather than from algorithmic novelty, and we expect that to remain true of most real demand-forecasting problems of this kind.

VIII. Acknowledgment

We thank Abdullah Gül University (AGU) and our instructor Dr. Khaled Hejja for supporting this work and for the COMP468 course material that this project is built on. This work was conducted as part of COMP468 at Abdullah Gül University.

IX. Kaggle Submission Evidence

Competition: "A Cloned Airbnb Prediction Competition – K353." URL: https://www.kaggle.com/competitions/a-cloned-airbnb-booking-prediction-competition-k-353. Team name: Bakır-Başkal. Best leaderboard score: 185.499 MSE (rank 1 of 6 teams, 9 submissions). Most recent submission: 185.642 MSE. The screenshot of the leaderboard at the time of writing is included with the submission package.

Scope statement. Models used in the submitted solution are restricted to COMP468 course material: Linear Regression, Random Forest, Gradient Boosting, XGBoost, and a PyTorch MLP trained with Adam, combined by a convex weighted blend. No external models (e.g. CatBoost or LightGBM) were used. We did not use Kaggle notebooks or Colab for training; we used GitHub and a local IDE, and uploaded submission `.csv` files to Kaggle manually.

X. Complete Feature Inventory

Here is the complete inventory of all 149 engineered features, along with the identifier and target variables, and their corresponding data types.

Feature Name	Type	Feature Name	Type	Feature Name	Type
PropertyID	integer (ID)	q1_days_A	binary	h1_available_rate	float
NumReserveDays2016Q3	integer (target)	q1_days_B	binary	h1_reserved_rate	float
PropertyType	str	q1_days_R	binary	h1_price_range	float
ListingType	str	q1_unique_reservations	integer	h1_price_cv	float
Zipcode	integer	q1_price_mean	float	h1_price_iqr	float
Neighborhood	str	q1_price_median	float	q1_we_days	integer
MetropolitanStatisticalArea	str	q1_price_std	float	q1_we_booked	binary
NumberofReviews	float	q1_price_nunique	integer	q1_we_price_mean	float
OverallRating	float	q1_price_min	float	q1_wd_days	integer

Feature Name	Type	Feature Name	Type	Feature Name	Type
Bedrooms	float	q1_price_max	float	q1_wd_booked	binary
Bathrooms	float	q1_price_q25	float	q1_wd_price_mean	float
MaxGuests	integer	q1_price_q75	float	q1_we_booking_rate	float
ResponseRate	float	q1_booking_rate	float	q1_wd_booking_rate	float
ResponseTimemin	float	q1_available_rate	float	q1_weekend_premium	float
Superhost	float	q1_reserved_rate	float	q1_weekend_minus_weekday_book_rate	float
CancellationPolicy	str	q1_price_range	float	q2_we_days	integer
SecurityDeposit	float	q1_price_cv	float	q2_we_booked	binary
CleaningFee	float	q1_price_iqr	float	q2_we_price_mean	float
ExtraPeopleFee	float	q2_days_total	integer	q2_wd_days	integer
PublishedNightlyRate	integer	q2_days_A	binary	q2_wd_booked	binary
PublishedMonthlyRate	float	q2_days_B	binary	q2_wd_price_mean	float
PublishedWeeklyRate	float	q2_days_R	binary	q2_we_booking_rate	float
MinimumStay	integer	q2_unique_reservations	integer	q2_wd_booking_rate	float
NumberofPhotos	float	q2_price_mean	float	q2_weekend_premium	float
BusinessReady	float	q2_price_median	float	q2_weekend_minus_weekday_book_rate	float
InstantbookEnabled	float	q2_price_std	float	q1_booked_m1	binary
Latitude	float	q2_price_nunique	integer	q1_booked_m2	binary
Longitude	float	q2_price_min	float	q1_booked_m3	binary
PropertyAgeDays	integer	q2_price_max	float	q2_booked_m4	binary
ReviewsPerMonth	float	q2_price_q25	float	q2_booked_m5	binary
PricePerGuest	float	q2_price_q75	float	q2_booked_m6	binary
WeeklyDiscount	float	q2_booking_rate	float	q1_status_changes	integer
MonthlyDiscount	float	q2_available_rate	float	q1_new_bookings	integer
BedBathRatio	float	q2_reserved_rate	float	q1_booking_ends	integer
ResponseRate_missing	binary	q2_price_range	float	q2_status_changes	integer
ResponseTimemin_missing	binary	q2_price_cv	float	q2_new_bookings	integer
SecurityDeposit_missing	binary	q2_price_iqr	float	q2_booking_ends	integer
CleaningFee_missing	binary	h1_days_total	integer	h1_status_changes	integer
ExtraPeopleFee_missing	binary	h1_days_A	binary	h1_new_bookings	integer
OverallRating_missing	binary	h1_days_B	binary	h1_booking_ends	integer
HasReviews	binary	h1_days_R	binary	recent_days	float
kw_luxury	binary	h1_unique_reservations	integer	recent_booked	binary
kw_private	binary	h1_price_mean	float	recent_available	float
kw_near	binary	h1_price_median	float	recent_price_mean	float

Feature Name	Type	Feature Name	Type	Feature Name	Type
	ry				
kw_cozy	binary	h1_price_std	float	recent_price_std	float
kw_view	binary	h1_price_nunique	integer	recent_booking_rate	float
kw_modern	binary	h1_price_min	float	recent_available_rate	float
title_len	integer	h1_price_max	float	price_change_q2_q1	float
title_words	integer	h1_price_q25	float	booking_rate_change_q2_q1	float
geo_cluster	str	h1_price_q75	float		
q1_days_total	integer	h1_booking_rate	float		

[1] C. Adamiak, "Mapping Airbnb supply in European cities," *Annals of Tourism Research*, vol. 71, pp. 67–71, 2018, doi: 10.1016/j.annals.2018.02.008.

[2] U. Gunter and I. Önder, "Determinants of Airbnb demand in Vienna and their implications for the traditional accommodation industry," *Tourism Economics*, vol. 24, no. 3, pp. 270–293, 2018, doi: 10.1177/1354816617731196.

[3] D. Wang and J. L. Nicolau, "Price determinants of sharing economy-based accommodation rental: A study of listings from 33 cities on Airbnb.com," *International Journal of Hospitality Management*, vol. 62, pp. 120–131, 2017, doi: 10.1016/j.ijhm.2016.12.007.

[4] J. Tang, J. Cheng, and M. Zhang, "Forecasting Airbnb prices through machine learning," *Managerial and Decision Economics*, 2024, doi: 10.1002/mde.3985.

[5] N. Camatti, G. di Tollo, G. Filograsso, and S. Ghilardi, "Predicting Airbnb pricing: A comparative analysis of artificial intelligence and traditional approaches," *Computational Management Science*, vol. 21, no. 30, 2024, doi: 10.1007/s10287-024-00513-9.

[6] P. Rezazadeh Kalehbasti, L. Nikolenko, and H. Rezaei, "Airbnb price prediction using machine learning and sentiment analysis," *arXiv preprint arXiv:1907.12665*, 2019.

[7] K353BA and M. Zhalechian, "A cloned Airbnb prediction competition – K353." Kaggle, 2026.

[8] D. Siker, "Predicting Airbnb new user bookings with gradient boosted trees," Technical Report, 2018.

[9] S. Chapman, S. Mohammad, and K. Villegas, "Predicting listing prices in dynamic short-term rental markets using machine learning models," *arXiv preprint arXiv:2308.06929*, 2023.

[10] Md. D. Islam, B. Li, K. S. Islam, R. Ahasan, Md. R. Mia, and Md. E. Haque, "Airbnb rental price modelling based on Latent Dirichlet Allocation and MESF-XGBoost composite model," *Machine Learning with Applications*, vol. 7, p. 100208, 2022, doi: 10.1016/j.mlwa.2021.100208.

[11] J. Tobin, "Estimation of relationships for limited dependent variables," *Econometrica*, vol. 26, no. 1, pp. 24–36, 1958, doi: 10.2307/1907382.

[12] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986, doi: 10.1007/BF00116251.

[13] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996, doi: 10.1007/BF00058655.

[14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[15] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

[16] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD int. Conf. Knowledge discovery and data mining*, 2016, pp. 785–794. doi: 10.1145/2939672.2939785.

[17] T. Hastie, R. Tibshirani, and J. Friedman, *The elements of statistical learning*, 2nd ed. New York: Springer, 2009. doi: 10.1007/978-0-387-84858-7.

[18] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[19] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[20] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998, doi: 10.1162/089976698300017197.

[21] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992, doi: 10.1016/S0893-6080(05)80023-1.

[22] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in neural information processing systems 32*, 2019, pp. 8024–8035.

[23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd int. Conf. Learning representations (ICLR)*, 2015.